# Turing Machines as Methods

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

# Language of a Turing Machine

- The language of a Turing machine $M$ denoted by $L(M)$ is the set of all strings accepted by the Turing machine
- Given a Turing machine $M$ and a string $w$
  - If $M$ accepts $w$, $w \in L(M)$
  - If $M$ rejects $w$, $w \notin L(M)$
  - If $M$ loops indefinitely on $w$, $w \notin L(M)$

# Language of a Method

- Lets think about Turing machines as Java methods with the following signature:

```
boolean method_name(String x)
```

- Lets
  - `true` in Java means **accept**
  - `false` in Java means **reject**
- Now, we can define the language of a method as **the set of all strings that the method returns** `true`

# Language of a Method

- Consider the following method

```
boolean M1(String x)
{
   return true;
}
```

- What is the language of the above method?
  - Obviously, `M1()` returns `true` no matter what argument of type `String` you use
  - Thus, the language of the method `M1()` is $\Sigma^*$
- The above method is pretty much the same as the Turing machine below:

$M_1 =$ "On input $x$:

1. *accept*"

- Note that $L(M_1) = \Sigma^*$

# Language of a Method

- Consider the following method

```
boolean M2(String x)
{
    return false;
}
```

- What is the language of the above method?
  - Obviously, `M2()` returns `false` no matter what argument of type `String` you use
  - Thus, the language of the method `M2()` is $\emptyset$
- The above method is pretty much the same as the Turing machine below:

$M_2 =$ "On input $x$:

1. *reject*"

- Note that $L(M_2) = \emptyset$

# Language of a Method

- Consider the following method

```
boolean M3(String x)
{
    if(x.equals(""))
        return true;
    else
        return false;
}
```

- What is the language of the above method?
  - The above method only returns `true` if we call use the empty string as the argument
  - For any other arguments that are not the empty string, it will always return `false`
  - Thus, the language of the method `M3()` is $\{\varepsilon\}$

# Language of a Method

- The method `M3()` is pretty much the same as the Turing machine below:

  $M_3 =$ "On input $x$:

  1. If $x = \varepsilon$, *accept*
  2. If $x \neq \varepsilon$, *reject*

- Note that $L(M_3) = \{\varepsilon\}$

# Language of a Method

- Suppose we have the method named `is_0n1n` with the following signature:

```
boolean is_0n1n(String s)
```

  - This method returns `true` if the string `s` is in the form of $0^n1^n$ for any $n \geq 0$
  - It returns `false` for any other strings
  - Examples:
    - `is_0n1n("")` is `true`
    - `is_0n1n("0101")` is `false`
    - `is_0n1n("01")` is `true`
    - `is_0n1n("00111")` is `false`
    - `is_0n1n("000111")` is `true`

# Language of a Method

- Consider the following method

```
boolean M4(String x)
{
    boolean p = is_0n1n(x);
    if(p)
        return true;
    else
        return false;
}
```

- What is the language of the above method?
  - Obviously, `M4()` returns `true` for any argument of type `String` of the form $0^n1^n$ for any $n \geq 0$
  - For any other strings, it will return `false`
  - Thus, the language of the method `M4()` is $\{0^n1^n \mid n \geq 0\}$

# Language of a Method

- The method `M4()` is pretty much the same as the Turing machine below:

  $M_4 =$ "On input $x$:

  1. If $x$ is in the form of $0^n1^n$, *accept*
  2. If $x$ is not in the form of $0^n1^n$, *reject*

- The language of the above machine is

$$L(M_4) = \{0^n1^n \mid n \geq 0\}$$

# Language of a Method

- Consider the following method

```
boolean M5(String x)
{
    while(true);
    return true;
}
```

- What is the language of the above method?
  - Obviously, no matter what argument of type `String` you use when call the above method, it will not return
  - It loops indefinitely on all arguments.
  - Thus, the language of the method `M5()` is $\emptyset$
- The above method `M5()` is pretty much the same as the Turing machine below:

$M_5 =$ "On input $x$:

1. Enter infinite loop
2. *accept*"

- Note that $L(M_5) = \emptyset$

# Language of a Method

- Now suppose we have a method named `M()` and and a string named `w` as follows:

```
boolean M(String s) {...}
String w = "...";
```

- Note that we have no idea what is the method `M()`
  - It just a method that takes an argument of type `String` and returns `boolean`
  - Just imagine that it can be any method with the above signature
  - Can even thing of `M` as a reference variable that refers to a method of the above signature
- Obviously, `w` is a variable of type `String`.
- What if we call `M(w)`?
  - We do not know whether it will return `true`, or `false`
  - It may not return at all (loops indefinitely)
  - We have no idea what are `M` and `w`

# Language of a Method

- Consider the following method

```
boolean N1(String x)
{
    boolean p = M(w);
    if(p)
        return true;
    else
        return false;
}
```

- In this case, we have no idea about the language of the method `N1()`
- This is because have no idea what does `M(w)` return when called
  - It may return `true`
  - It may return `false`
  - It may not return at all (`M(w)` loops indefinitely)

# Language of a Method

- Suppose `M(w)` returns `true`, we can replace `M(w)` by `true` as shown below:

```
boolean N1(String x)
{
    boolean p = true;       // Used to be boolean p = M(w);
    if(p)
        return true;
    else
        return false;
}
```

- Now, what is the language of the above method `N1`?
  - The language of the method `N1` is $\Sigma^*$
  - This is under the assumption that `M(w)` returns `true`
- So, here is what we can say about the method `N1()` so far:
  - If `M(w)` returns `true`, the language of `N1()` is $\Sigma^*$

# Language of a Method

- Suppose `M(w)` returns `false`, we can replace `M(w)` by `false` as shown below:

```
boolean N1(String x)
{
    boolean p = false;      // Used to be boolean p = M(w);
    if(p)
        return true;
    else
        return false;
}
```

- Now, what is the language of the above method `N1`?
  - The language of the method `N1()` is $\emptyset$
  - This is under the assumption that `M(w)` returns `false`
- So, here is what we can say about the method `N1()` so far:
  - If `M(w)` returns `true`, the language of `N1()` is $\Sigma^*$
  - If `M(w)` returns `false`, the language of `N1()` is $\emptyset$

# Language of a Method

- Suppose `M(w)` does not return at all
  - It loops indefinitely when called
- Since `N1()` calls `M(w)` and `M(w)` loops indefinitely, `N1()` also loops indefinitely.
- In this case, we can insert the statement `while(true);` as shown below

```
boolean N1(String x)
{
    while(true);          // Inserted to make N1 loops indefinitely
    boolean p = M(w);
    if(p)
        return true;
    else
        return false;
}
```

- Now, what is the language of the above method `N1`?
  - Since `N1()` does not return `true` on any argument of type `String`
    - The language of the method `N1` is $\emptyset$
  - This is under the assumption that `M(w)` loops indefinitely

# Language of a Method

- So, here is what we can say about the method `N1()` below:

```
boolean N1(String x)
{
    boolean p = M(w);
    if(p)
        return true;
    else
        return false;
}
```

  - If `M(w)` returns `true`, the language of `N1()` is $\Sigma^*$
  - If `M(w)` returns `false`, the language of `N1()` is $\emptyset$
  - If `M(w)` loops indefinitely, the language of `N1()` is $\emptyset$
- The behavior of the methods `N1()` depends on what happen when `M(w)` is called
  - But again, we have no idea what are `M()` and `w`

# Language of a Method

- Given a Turing machine $M$ and a string $w$, the method `N1()` is pretty much the same as the Turing machine below:

  $N_1$ = "On input $x$:

  1. Run $M$ on input $w$
  2. If $M$ accepts $w$, *accept*
     If $M$ rejects $w$, *reject*"

- Similarly, we have no idea what is the language of $N_1$ but at least we know that
  - If $M$ accepts $w$, $L(N_1) = \Sigma^*$
  - If $M$ rejects $w$, $L(N_1) = \emptyset$
  - If $M$ loops indefinitely on $w$, $L(N_1) = \emptyset$
- We can also conclude that
  - If $M$ accepts $w$, the language of $N_1$ is not empty
  - If $M$ does not accept $w$, the language of $N_1$ is empty